Real Estate AI Platform

A Project-II Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Ayush Jain (EN22CS301247)

Ayush Patidar (EN22CS301253)

Bhumika Choyal (EN22CS301276)

Cherry Mehta (EN23CS301292)

Diya Goyal (EN22CS301351)

Under the Guidance of

Prof. Vineeta Rathore
Prof. Maya Yadav Baniya
Prof. Shyam Patel

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

JAN-JUNE 2026

Real Estate AI Platform

A Project-II Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Ayush Jain (EN22CS301247)

Ayush Patidar (EN22CS301253)

Bhumika Choyal (EN22CS301276)

Cherry Mehta (EN23CS301292)

Diya Goyal (EN22CS301351)

Under the Guidance of

Prof. Vineeta Rathore
Prof. Maya Yadav Baniya
Prof. Shyam Patel

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

JAN-JUNE 2026

Report Approval

The project work "**Real Estate AI Platform**", is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn therein; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner

Name:

Designation

Affiliation

External Examiner

Name:

Designation

Affiliation

Declaration

We hereby declare that the project entitled "**Real Estate AI Platform**" submitted in partial fulfillment for the award of the degree of Bachelor of Technology in 'Computer Science and Engineering' completed under the supervision of **Prof. Vineeta Rathore, Prof. Maya Yadav Baniya and Prof.Shyam Patel**, Faculty of Engineering, Medicaps University Indore is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the students with date

Ayush Jain (EN22CS301247)

Ayush Patidar (EN22CS301253)

Bhumika Choyal (EN22CS301276)

Cherry Mehta (EN22CS301292)

Diya Goyal (EN22CS301351)

Certificate

We, **Prof. Vineeta Rathore, Prof. Maya Yadav Baniya, and Prof. Shyam Patel** certify that the project entitled "Real Estate AI Platform" submitted in partial fulfillment for the award of the degree of Bachelor of Technology of Computer Applications by **Ayush Jain, Ayush Patidar, Bhumika Choyal, Cherry Mehta and Diya Goyal** is the record carried out by them under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof. Vineeta Rathore

Prof. Maya Yadav

Prof. Shyam Patel

Name of Internal Guide

Computer Science & Engineering

Medicaps University, Indore

Name of External Guide

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

Acknowledgements

I would like to express my deepest gratitude to the Honorable Chancellor, **Shri R C Mittal**, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Medicaps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Prof. (Dr.) Ratnesh Litoriya,** Associate Dean of Computing & Allied Branches, Medicaps University, for giving me a chance to work on this project. I would also like to thank my Head of Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

I express my heartfelt gratitude to my External Guide **Mr. Suraj, Mr. Ashwin, Mr. Vaibhav** as well as to my Internal Guide**, Prof. Vineeta Rathore, Prof. Maya Yadav Baniya, and Shyam Patel**, Department of Computer Science Engineering, Medicaps University, without whose continuous help and support, this project would ever have reached to the completion.

I would also like to thank my team who extended their kind support and help towards the completion of this project. It is their help and support, due to which we became able to complete the design and technical report.
Without their support this report would not have been possible.

Ayush Jain (EN22CS301247)
Ayush Patidar (EN22CS301253)
Bhumika Choyal(EN22CS301276)
Cherry Mehta (EN22CS301292)
Diya Goyal (EN23CS301351)

B.Tech. IV Year
Department of Computer Science & Engineering
Faculty of Engineering
Medicaps University, Indore

Abstract

The rapid advancement of Artificial Intelligence has enabled the development of intelligent systems capable of transforming domain-specific learning and communication workflows. This project presents an integrated Real Estate AI system that combines an interactive educational bot with a production-grade triage agent, supported by automated infrastructure provisioning.

The educational component leverages Large Language Models (LLMs) via the Groq API to generate dynamic, property listing-based assessments, evaluate user responses, and provide detailed contextual explanations. This creates a personalized feedback loop that enhances user understanding of real estate concepts.

In parallel, the system incorporates a reactive triage agent designed to process real-time real estate communications. The agent performs intent classification and urgency detection, applies Named Entity Recognition (NER) to extract critical information such as property identifiers and dates, and generates context-aware draft responses using LLM capabilities. This significantly improves response efficiency and reduces manual workload in communication pipelines. To ensure scalability and consistency, the project utilizes Infrastructure as Code (IaC) practices through tools such as Terraform and Ansible for automated deployment of local and cloud-based environments. Containerization using Docker further enables reproducible and portable system setup.

The integrated architecture demonstrates a modular yet cohesive design, combining intelligent learning, automated decision-making, and scalable infrastructure, thereby providing a robust solution for modern real estate applications.

Keywords:

- Generative AI
- Multi-Agent System
- Real Estate System
- React
- Python
- Groq API
- Docker
- Kubernetes
- DevOps
- Artificial Intelligence

Table of Contents

List of Figures

List of Tables

Abbreviations

S. No	Abbreviations	Full Form
1	AI	Artificial Intelligence
2	Gen AI	Generative Artificial Intelligence
3	LLM	Large Language Model
4	API	Application Programming Interface
5	UI	User Interface
6	UX	User Experience
7	ML	Machine Learning
8	NLP	Natural Language Processing
9	DB	Database
10	AWS	Amazon Web Services
11	CPU	Central Processing Unit
12	GPU	Graphics Processing Unit
13	K8s	Kubernetes
14	CLI	Command Line Interface
15	EC2	Elastic Compute Cloud

CHAPTER 1

INTRODUCTION

1.1. Introduction

The rapid advancement of Artificial Intelligence (AI) has significantly transformed how industries manage communication, decision-making, and user engagement. In the Real Estate domain, organizations frequently deal with high volumes of unstructured communication such as property inquiries, client requests, booking confirmations, and complaint handling. Manual processing of such communication is time-consuming, error-prone, and inefficient, particularly when timely responses are critical for customer satisfaction and business operations.

The proposed system, titled **"AI-Driven Real Estate Triage and Learning Platform"**, addresses these challenges by integrating Agentic AI, Generative AI, and DevOps practices into a unified solution. The system includes a **reactive triage agent** that automates the classification of incoming real estate communications based on urgency and intent. It further extracts key entities such as property IDs, dates, client names, and transaction details using Named Entity Recognition (NER), and generates context-aware draft responses using a Large Language Model (LLM).

In addition to communication automation, the system incorporates an **AI-driven educational bot** tailored for the Real Estate domain. This module engages users through scenario-based assessments derived from property listings, evaluates responses intelligently, and provides detailed contextual explanations. This creates a personalized learning loop that enhances domain understanding and decision-making skills.

The backend of the system is implemented using FastAPI (Python 3.11), while the frontend is developed using React (Vite) to ensure an interactive and responsive user experience. The

system is containerized using Docker and deployed on cloud infrastructure such as AWS EC2. Infrastructure provisioning and configuration are automated using Terraform and Ansible, ensuring scalability, repeatability, and minimal manual intervention. CI/CD pipelines using GitHub Actions further enhance deployment efficiency and reliability.

1.2. Literature Review

Recent developments in Artificial Intelligence, particularly in Natural Language Processing (NLP), have enabled significant improvements in text classification, entity extraction, and automated content generation. Large Language Models (LLMs) have proven effective in understanding contextual semantics, enabling intelligent decision-making in domains such as customer support, healthcare, and real estate.

Studies on Named Entity Recognition (NER) highlight its effectiveness in extracting structured information from unstructured text, which is crucial for applications involving document processing and communication automation. In real estate, this includes extracting property identifiers, pricing details, locations, and timelines from client interactions.

Furthermore, research on multi-agent systems demonstrates how complex workflows can be decomposed into specialized agents responsible for tasks such as classification, reasoning, and response generation. This approach improves system modularity, scalability, and maintainability.

In the educational domain, Generative AI has enabled the creation of adaptive learning systems. Retrieval-Augmented Generation (RAG) has been particularly impactful in improving response accuracy by grounding outputs in relevant contextual data, thereby reducing hallucination in LLM responses.

Additionally, the adoption of DevOps practices such as containerization, Infrastructure as Code (IaC), and CI/CD pipelines has streamlined the deployment of AI systems. These practices ensure reproducibility, scalability, and efficient lifecycle management of applications.

Despite these advancements, many existing systems lack integration between intelligent communication handling, domain-specific learning, and automated deployment. The proposed system addresses this gap by combining triage automation, AI-based learning, and DevOps into a single Real Estate platform.

1.3. Objectives

The primary objectives of the proposed system are:

- To develop a reactive triage agent capable of classifying real estate communications based on urgency and intent.
- To implement Named Entity Recognition (NER) for extracting structured data such as property IDs, dates, client names, and pricing details.

- To generate context-aware draft responses using a Large Language Model for efficient communication handling.
- To design an AI-driven educational bot that generates property-based assessment questions.
- To evaluate user responses and provide detailed contextual explanations for incorrect answers.
- To implement Retrieval-Augmented Generation (RAG) for improving response accuracy.
- To integrate triage and learning modules into a unified platform.
- To develop a responsive frontend using React (Vite).
- To build scalable backend services using FastAPI.
- To containerize the system using Docker for consistent deployment.
- To automate infrastructure provisioning using Terraform and configuration management using Ansible.
- To implement CI/CD pipelines using GitHub Actions.
- To ensure real-time processing with low latency.
- To design a modular and extensible system architecture.

1.4. Significance

The proposed system provides significant value in modernizing Real Estate operations by automating communication workflows and enhancing user learning experiences. The triage agent reduces manual workload by automatically classifying and responding to client communications, ensuring faster response times and improved customer satisfaction.

The educational bot adds an innovative dimension by enabling users to learn through interactive assessments based on real-world property scenarios. The detailed feedback mechanism helps users understand mistakes and strengthens domain knowledge.

From a technical perspective, the integration of AI with DevOps ensures scalability, reliability, and reproducibility of the system. This makes the platform suitable for real-world deployment in real estate agencies, property management firms, and online property marketplaces.

1.5. Research Design

The research design follows a system-oriented and applied approach focused on solving real-world challenges in the Real Estate domain. The system is designed using a modular architecture where components such as classification, entity extraction, response generation, and quiz evaluation operate as independent yet interconnected modules.

A multi-agent framework is used to distribute tasks among specialized agents, improving efficiency and maintainability. The backend is developed using FastAPI to handle API-driven communication, while the frontend uses React (Vite) to provide an interactive interface.

Large Language Models are integrated via API for intelligent processing, and spaCy is used for Named Entity Recognition. The system follows an iterative development approach, where each module is designed, tested, and optimized.

Deployment is handled using Docker for containerization, Terraform for infrastructure provisioning, Ansible for configuration management, and GitHub Actions for CI/CD. This ensures scalability, consistency, and ease of maintenance.

1.6. Source of Data

The system primarily uses dynamically generated and user-provided data. For the triage module, input consists of real estate communication scenarios such as property inquiries, booking requests, complaints, and negotiation messages.

For the educational module, data is generated dynamically using LLMs based on property listings, pricing scenarios, and real estate concepts. Contextual prompts are used to guide question generation and evaluation.

Pre-trained NLP models such as spaCy are used for entity extraction, and LLMs are used for reasoning and generation tasks. No proprietary datasets are required, ensuring compliance with ethical standards.

1.7. Chapter Scheme

- **Chapter 1 — Introduction:** Overview, objectives, and system significance in the Real Estate domain.
- **Chapter 2 — Requirement Specification:** Functional and non-functional requirements, system environment, and constraints.
- **Chapter 3 — System Design:** Architecture, data flow diagrams, and module design.
- **Chapter 4 — Implementation and Testing:** Technologies used, implementation details, and testing strategies.
- **Chapter 5 — Results and Discussion:** System outputs and performance evaluation.
- **Chapter 6 — Summary and Conclusions:** Summary of outcomes and effectiveness.
- **Chapter 7 — Future Scope:** Enhancements and scalability improvements.

CHAPTER 2

REQUIREMENTS SPECIFICATION

2.1. Experimental Set-up

The experimental set-up of the proposed **AI-Driven Real Estate Triage and Learning Platform** defines the environment used for development, testing, and deployment of the system. The configuration ensures consistency, scalability, and reproducibility across different environments.

The system is designed using a modern full-stack architecture integrating AI-based processing with frontend and backend technologies. The setup supports both local development and cloud-based deployment.

2.1.1. Hardware Configuration

- **Processor:** Intel Core i3/i5/i7 or equivalent
- **RAM:** Minimum 4 GB (8 GB recommended)
- **Storage:** At least 20 GB free disk space
- **Internet Connectivity:** Required for API access and cloud deployment

2.1.2. Software Environment

- **Operating System:** Windows 10 / Linux / macOS
- **Backend:** Python 3.11 with FastAPI
- **Frontend:** React.js (Vite)
- **Libraries/Tools:** CrewAI, spaCy, litellm, LLM API (Groq/OpenAI)
- **Containerization:** Docker
- **Cloud Platform:** AWS EC2 (or equivalent)
- **Infrastructure Tools:** Terraform, Ansible
- **CI/CD:** GitHub Actions
- **Development Tools:** Visual Studio Code, Web Browser

2.1.3. Development Workflow

The development process follows a structured and iterative approach:

- Requirement analysis and system planning
- Designing system architecture and workflows
- Backend development (API creation and AI integration)
- Frontend development using React (Vite)
- Integration of frontend and backend components
- Testing and validation of system functionalities
- Containerization using Docker
- Deployment using Terraform, Ansible, and CI/CD pipelines

2.1.4. Testing Environment

The system is tested in both **local** and **cloud environments**:

- Locally, the FastAPI backend runs on a development server and interacts with the React frontend.
- Functional testing is conducted for:
  - Message classification
  - Entity extraction
  - Draft response generation
  - Quiz generation and evaluation
- Cloud testing is performed on AWS EC2 to validate:
  - API performance
  - Real-time response handling
  - Deployment stability
  - Cross-origin request handling

● Procedures Adopted

The system development follows a modular and iterative methodology:

Initially, the Real Estate domain was analyzed to identify key requirements such as automated communication handling and domain-specific learning. Based on this, the system was divided into two major modules:

1. **Triage Agent Module**
2. **Educational Quiz Module**

The backend was developed using FastAPI to ensure efficient API handling, while the frontend was built using React (Vite) for an interactive user experience.

A multi-agent architecture was implemented using CrewAI, where:

- One agent handles classification (urgency and intent)
- One agent performs Named Entity Recognition (NER)
- One agent generates draft responses

The educational module uses LLMs to generate questions and evaluate answers with contextual explanations.

Continuous testing and validation were performed during development. The system was containerized using Docker and deployed on AWS EC2. Infrastructure automation was achieved using Terraform and Ansible, with CI/CD pipelines implemented via GitHub Actions.

2.3. Functional Requirements

The functional requirements define the core operations of the system.

Table 2.1 Functional Requirements

Requirement ID	Requirement Description
FR01	The system shall accept user input in the form of real estate communication messages.
FR02	The system shall classify messages based on urgency and intent using an AI-driven triage agent.
FR03	The system shall extract entities such as property IDs, dates, client names, and pricing details using NER. .
FR04	The system shall generate context-aware draft responses using a Large Language Model.
FR05	The system shall generate quiz questions based on property listings and real estate scenarios.

FR06	The system shall evaluate user responses and provide scores with contextual explanations.
FR07	The system shall provide an interactive frontend interface for triage and quiz functionalities.
FR08	The system shall expose RESTful APIs for communication between frontend and backend.
FR09	The system shall support deployment in a containerized environment using Docker.

2.4. Non-Functional Requirements

The non-functional requirements define the quality attributes of the system.

Table 2.2 Non-Functional Requirements

Requirement ID	Requirement Description
NFR01	The system shall provide real-time or near real-time responses.
NFR02	The system shall ensure high availability and reliability.
NFR03	The system shall maintain modularity for easy maintenance.
NFR04	The system shall provide structured and consistent output formatting.
NFR05	The system shall support cross-origin requests for frontend-backend communication.
NFR06	The system shall be deployable using Docker containers.
NFR07	The system shall ensure secure handling of API keys and credentials.

NFR08	The system shall provide a responsive and user-friendly interface.
NFR09	The system shall be compatible with standard web browsers and operating systems

2.5. Constraints

The system is developed under the following constraints:

- Dependence on external LLM APIs may introduce latency and availability issues.
- Output accuracy depends on pre-trained model performance.
- The system does not include persistent database storage in the current version.
- Only text-based inputs are supported (no image/audio processing).
- Deployment relies on cloud infrastructure such as AWS EC2.
- The system primarily supports English language processing.
- Hardware limitations may affect local development performance.

CHAPTER 3

SYSTEM DESIGN

3.1. Function Oriented Design for procedural approach

The proposed **AI-Driven Real Estate Triage and Learning Platform** follows a function-oriented design approach, where the entire system is structured into a sequence of well-defined procedures. Each functionality is implemented as an independent module, ensuring clear separation of concerns and ease of maintenance.

The system processes user input through a structured pipeline:

- In the **triage module**, the workflow begins with user input (real estate communication), followed by classification of urgency and intent, extraction of relevant entities such as property IDs and dates, and generation of a context-aware draft response using an LLM.
- In the **educational module**, the system generates property-based questions, accepts user responses, evaluates answers, and provides detailed feedback with explanations.

This procedural design ensures consistent data flow, modular implementation, and scalability for future enhancements.

3.1.1. Frontend Modules

The frontend is designed using React (Vite) and consists of the following modules:

- **User Interface Module:**
  Provides the overall layout and navigation using headers, menus, and tab-based navigation for triage and quiz features.
- **Triage Interface Module:**
  Handles user input of real estate messages and displays outputs including:
  - Message classification (urgency and intent)
  - Extracted entities (property ID, price, location, etc.)
  - Generated draft responses
- **Quiz Interface Module:**
  Manages user interaction for learning:
  - Topic selection (e.g., pricing, property types)

- Question display
- Answer selection and evaluation

- **API Integration Module:**
  Communicates with backend APIs to send requests and receive processed data.
- **Routing Module:**
  Controls navigation between triage and quiz modules and initializes the application.

3.1.2. Backend Modules

The backend is developed using FastAPI and includes the following components:

- **API Management Module:**
  Handles HTTP requests and routes them to appropriate processing modules.
- **Triage Processing Module:**
  Implements the multi-agent workflow for:
  - Classification of urgency and intent
  - Named Entity Recognition
  - Draft response generation
- **Quiz Engine Module:**
  Responsible for:
  - Generating real estate-based questions
  - Evaluating user responses
  - Providing explanations and scores
- **NER Module (spaCy):**
  Extracts structured entities such as:
  - Property IDs
  - Dates
  - Prices
  - Locations
  - Client names
- **LLM Integration Module:**
  Connects with LLM APIs (Groq/OpenAI) for:
  - Text classification
  - Response generation
  - Answer evaluation

- **Utility Module:**
  Handles JSON parsing and ensures structured output formatting.

3.2. System Design

The system follows a **client–server architecture**:

- The **frontend (React)** interacts with users and sends requests.
- The **backend (FastAPI)** processes requests using AI modules.
- AI components (LLM + NER) perform intelligent processing.
- Results are returned to the frontend in structured format.

This design ensures:

- Scalability
- Real-time interaction
- Modular architecture
- Easy integration of future features

3.2.1. Data Flow Diagrams

A. Level 0 Data Flow Diagram

The Level 0 DFD provides a high-level overview of the system:

- The user interacts with the frontend interface.
- The frontend sends requests to the backend via APIs.
- The backend processes requests using triage or quiz modules.
- The processed output is returned to the frontend.

This represents the complete system as a single process.

Figure 3.1 Level 0 Data Flow Diagram

B. Level 1 Data Flow Diagram

The Level 1 DFD shows internal processing in detail:

Triage Flow:

1. User inputs real estate message
2. Backend receives request
3. Classification module determines urgency and intent
4. NER module extracts entities
5. LLM generates draft response
6. Output is returned to frontend

Quiz Flow:

1. User selects topic and difficulty
2. LLM generates question
3. User submits answer
4. Evaluation module processes response
5. Score and explanation are generated
6. Results are displayed

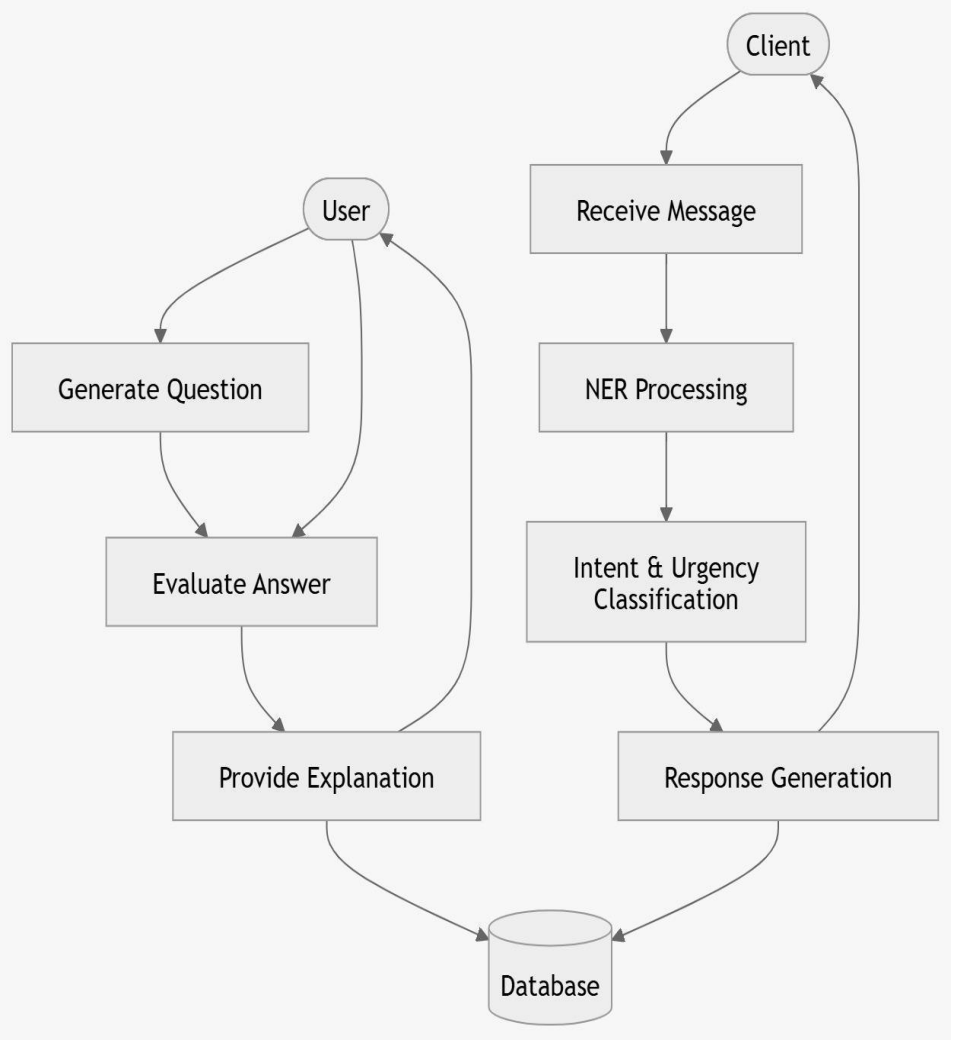

Figure 3.2 Level-1 Data Flow Diagram

3.2.2. Activity Diagram

The activity diagram represents the step-by-step workflow:

- User accesses the system
- Chooses either:
  - Triage module
  - Quiz module

Triage Workflow:

- Enter message
- Perform classification

- Extract entities
- Generate response
- Display results

Quiz Workflow:

- Select topic
- Generate question
- Submit answer
- Evaluate response
- Display score and explanation

The process ends after output is displayed.

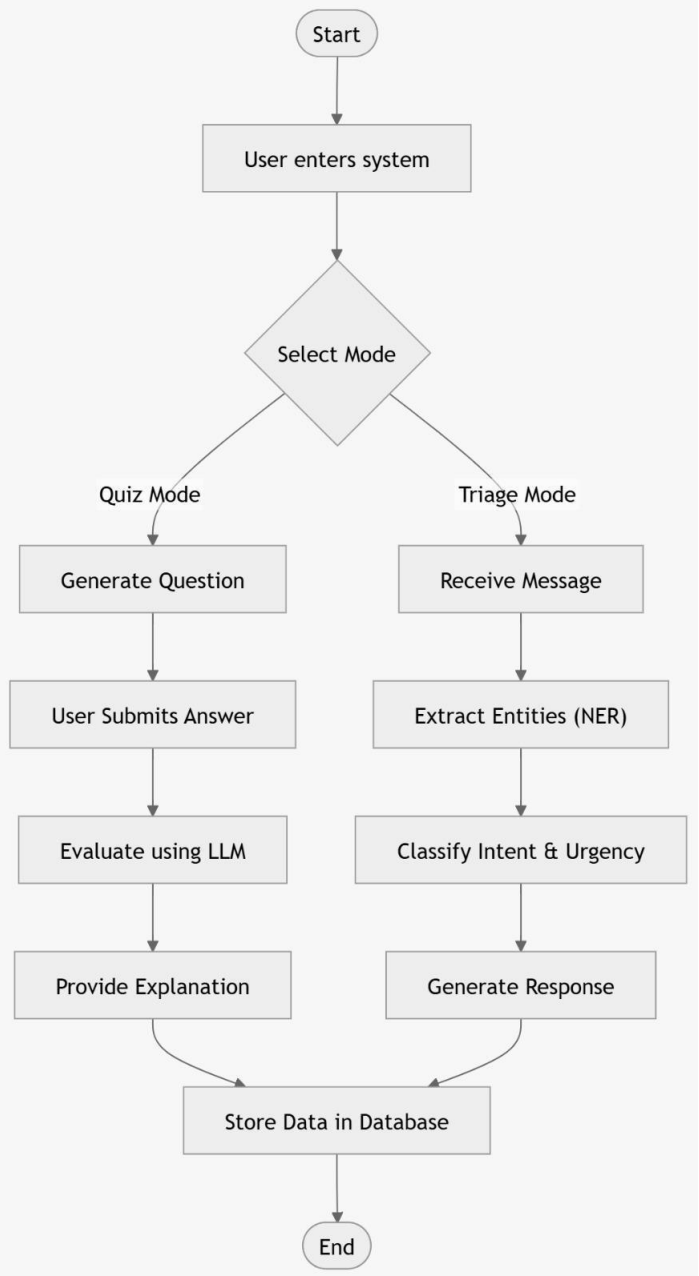

Figure 3.3 Activity Diagramchatgpt

3.2.3. Class Diagram

The class diagram defines system structure and relationships:

- **Frontend Classes:**
  - Header
  - TriagePanel
  - QuizPanel

- **APIClient Class:**
  Handles communication between frontend and backend
- **Backend Classes:**
  - FastAPIApp (main controller)
  - TriageAgent (coordinates workflow)
  - ClassifierAgent
  - NERAgent
  - DraftGenerator
  - QuizEngine
- **Supporting Classes:**
  - LLMService (handles API calls)
  - NERTool (entity extraction)
  - JSONParser (output formatting)

This structure ensures modularity and scalability.

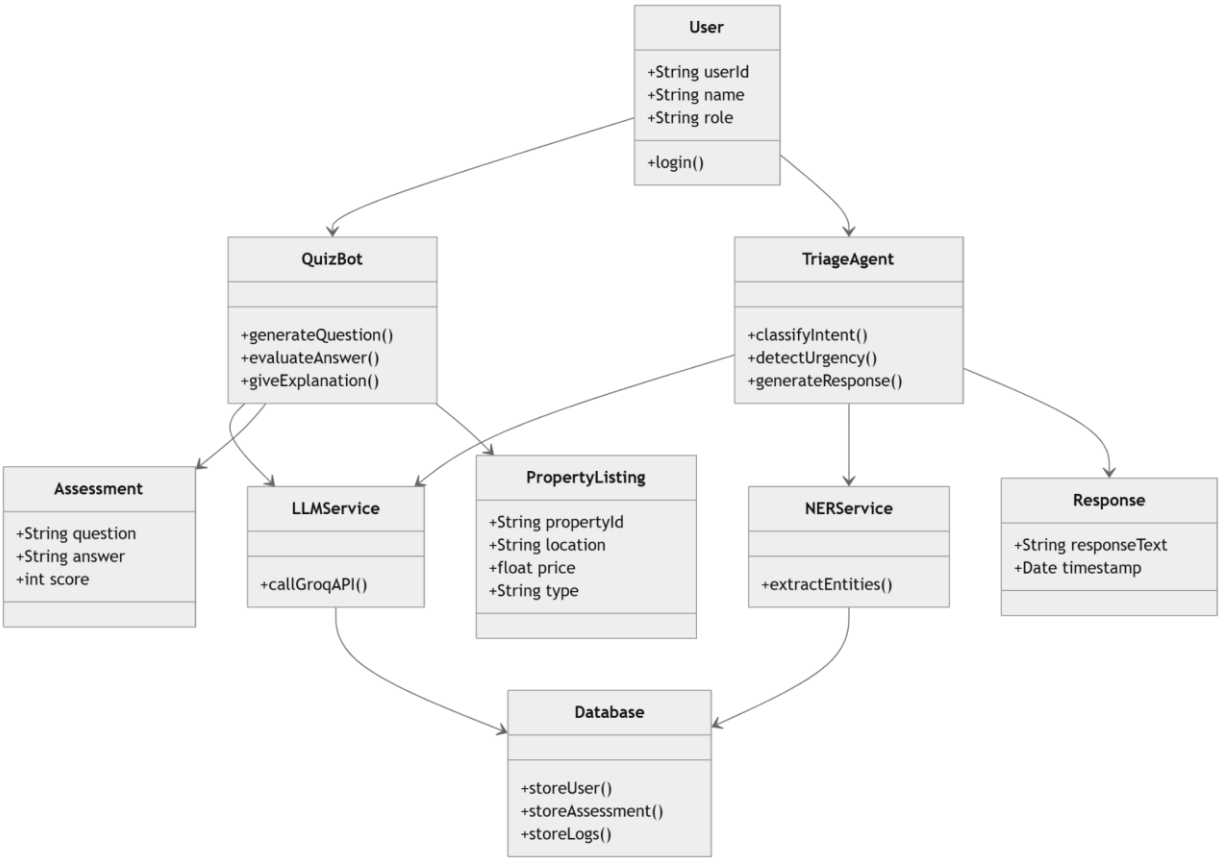

Figure 3.4 Class Diagram

3.2.4. ER Diagram

The ER diagram represents logical data relationships:

- **USER:** Interacts with system
- **TRIAGE_REQUEST:** Stores user message
- **TRIAGE_RESULT:** Contains classification and response
- **ENTITY:** Extracted data (property ID, price, etc.)
- **QUIZ_SESSION:** Represents quiz attempts
- **QUESTION:** Generated questions
- **ANSWER:** User responses and evaluation

Although no database is implemented, this logical structure defines data flow.

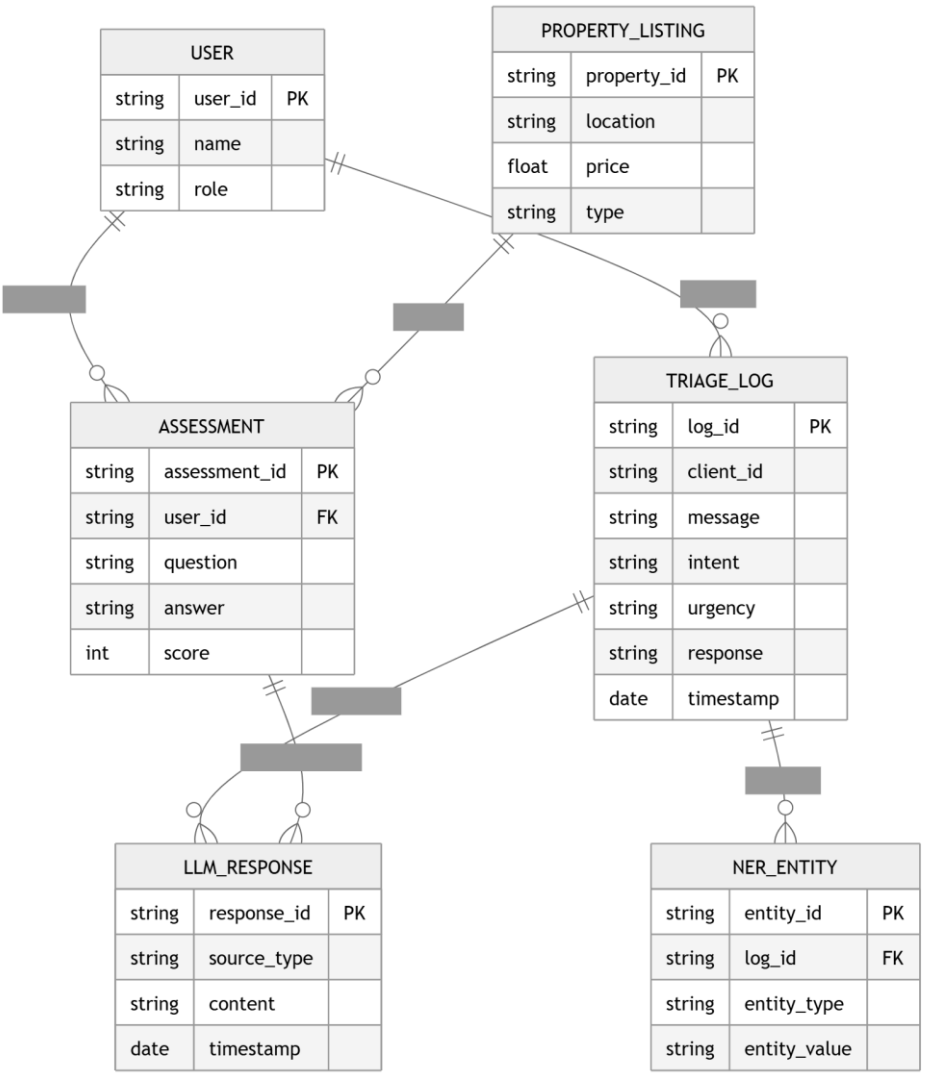

Figure 3.5 ER Diagram

3.2.5. Sequence Diagram

The sequence diagram shows interaction flow:

Triage Sequence:

1. User sends message
2. Frontend sends API request
3. Backend routes to triage agent
4. Classification agent processes input
5. NER extracts entities
6. LLM generates response
7. Backend returns result
8. Frontend displays output

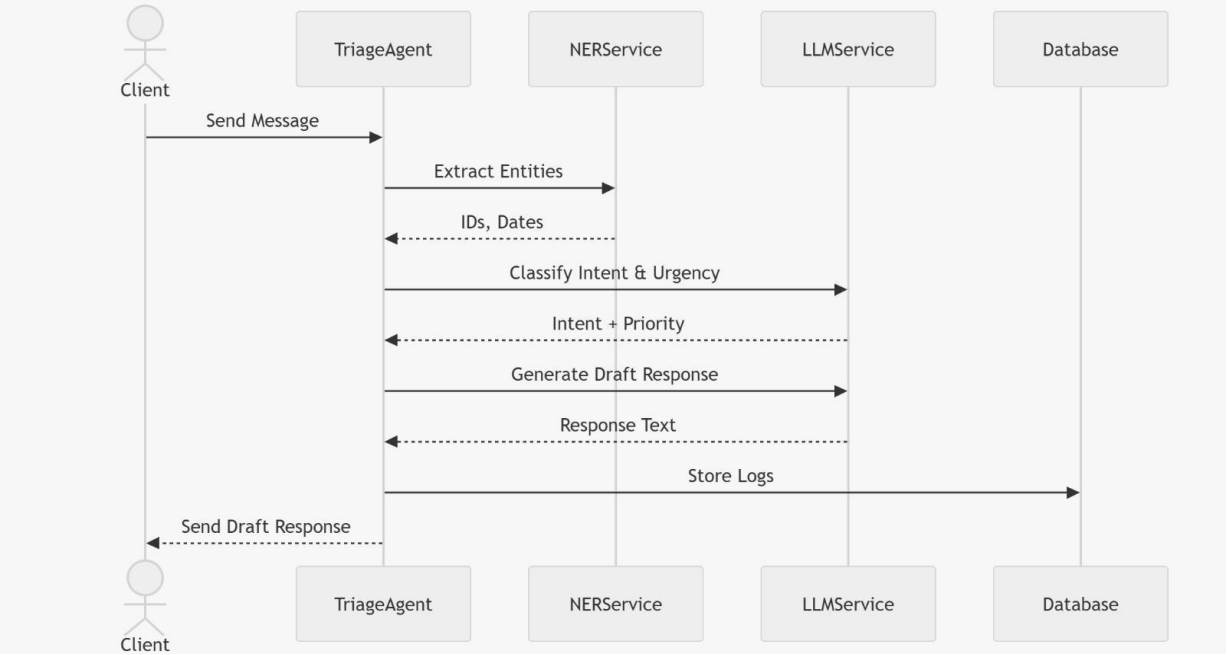

Figure 3.6 Sequence Diagram Triage Agent

Quiz Sequence:

1. User requests question
2. Backend generates question
3. User submits answer
4. Backend evaluates response
5. Results are returned
6. Frontend displays score and explanation

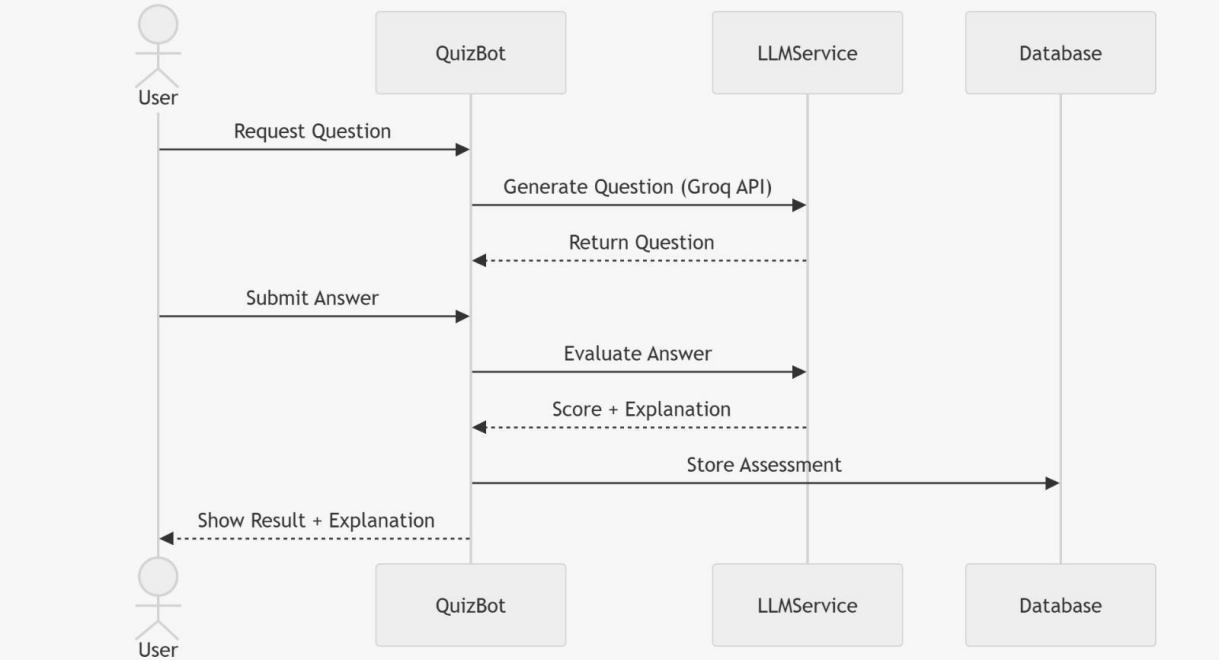

Figure 3.7 Sequence Diagram Quiz Bot

3.3. Database Design

The system does not use a traditional database but follows a **logical data structure**:

- Data is processed dynamically in real-time
- Information is stored temporarily in memory
- JSON format is used for data exchange

Logical Data Elements:

- User input messages
- Classification results
- Extracted entities
- Quiz questions
- Evaluation outputs

Advantages:

- Faster processing
- Reduced system complexity

- No dependency on database setup

This design ensures efficient and real-time operation of the system.

CHAPTER 4

IMPLEMENTATION AND TESTING

4.1. Introduction to Languages, IDE's, Tools and Technologies used for Implementation

The implementation of the proposed **AI-Driven Real Estate Triage and Learning Platform** follows a modern full-stack architecture that integrates Artificial Intelligence, web technologies, and DevOps practices. The system is designed to ensure scalability, modularity, and real-time performance.

The backend is developed using FastAPI, enabling high-performance API handling, while the frontend is built using React (Vite) to provide a responsive and interactive user interface. AI functionalities such as classification, entity extraction, and response generation are powered by Large Language Models (LLMs) and NLP tools.

4.1.1. Programming Languages

- **Python 3.11:**
  Used for backend development, API creation, and implementation of AI workflows including triage processing and quiz evaluation.
- **JavaScript (ES6+):**
  Used for frontend development with React to handle dynamic user interactions.
- **HTML5 and CSS3:**
  Used for structuring and styling the user interface to ensure responsiveness and usability.

4.1.2. Integrated Development Environments (IDEs)

- **Visual Studio Code (VS Code):**

  Used as the primary development environment for both frontend and backend. It provides debugging tools, extensions, and version control integration.

4.1.3. Tools and Technologies

FastAPI:
Used to develop RESTful APIs for handling triage processing and quiz operations.

React (Vite):

Used for building a fast and interactive frontend interface.

CrewAI:

Implements a multi-agent system where different agents handle classification, NER, and response generation.

spaCy:

Used for Named Entity Recognition (NER) to extract structured data such as property IDs, dates, prices, and locations.

LLM API (Groq / OpenAI):

Used for:

- Message classification
- Draft response generation
- Quiz generation and evaluation

litellm:

Acts as an interface to connect with LLM APIs efficiently.

Docker:

Used to containerize the application, ensuring consistent environments across development and deployment.

AWS EC2:

Used for cloud deployment of the application.

Terraform:

Used for Infrastructure as Code (IaC) to automate cloud resource provisioning.

Ansible:

Used for configuration management and automated setup of environments.

GitHub Actions:

Used to implement CI/CD pipelines for automated build and deployment.

Git & GitHub:

Used for version control and collaboration.

4.2 Testing Techniques and Test Plans

4.2.1. Testing Techniques Used

The system is tested using multiple testing approaches:

4.2.2. **Functional Testing (Black-box):**

Verifies system behavior based on input and output.

5.2.2. **Structural Testing (White-box):**

Tests internal logic, API routing, and agent workflows.

6.2.2. **Integration Testing:**

Ensures proper communication between frontend and backend.

7.2.2. **Unit Testing:**

Tests individual modules such as:

a. Classification module

b. NER module

c. Quiz engine

8.2.2. Test Plans

A structured test plan is designed to validate system performance and reliability.

Table 4.1 Test Cases

Test Case ID	Test Case Description	Input Data	Expected Output	Actual Output	Status
TC01	Triage Input Processing	Valid real estate message	Classification + entities + response	As Expected	Passed
TC02	Empty Input Handling	Empty message	Error message	As Expected	Passed
TC03	Entity Extraction	Message with property details	Entities extracted correctly	As Expected	Passed
TC04	Draft Response Generation	Valid inquiry	Context-aware response	As Expected	Passed
TC05	Quiz Generation	Topic + difficulty	Question generated	As Expected	Passed
TC06	Answer Evaluation	User answer	Score + explanation	As Expected	Passed

TC07	API Response	Valid API	Structured JSON	As	Passed

	Format	request	output	Expected	
TC08	Frontend Integration	API response	Data displayed correctly	As Expected	Passed

4.3. Installation and Access

The system can be accessed in two ways:

1. Cloud Deployment

- The application is deployed on AWS EC2
- Accessible via web browser using public IP
- No local setup required for end users

2. Local Setup

Steps to run locally:

1. Install dependencies:
    - Python 3.11
    - Node.js
    - Docker
2. Clone repository from GitHub
3. Configure environment variables (API keys)
4. Run backend:

    uvicorn main:app --reload

5. Run

    frontend:

    npm run dev

6. Access application via browser

CHAPTER 5

RESULTS AND DISCUSSION

5.1. User Interface Representation

The proposed **AI-Driven Real Estate Triage and Learning Platform** provides a clean, interactive, and user-friendly interface developed using React (Vite). The system ensures smooth navigation between modules and clear visualization of AI-generated outputs.

Triage Agent Interface

The triage interface allows users to input real estate-related communication such as:

- Property inquiries
- Booking requests
- Pricing negotiations
- Complaint messages

Output Displayed:

- **Classification Result:** Urgency (High/Medium/Low) and Intent (Inquiry, Complaint, Booking, etc.)
- **Extracted Entities:** Property ID, location, price, dates, client name
- **Draft Response:** AI-generated professional reply

The interface is designed in a structured card format to ensure clarity and readability of results.

Quiz Bot Interface

The quiz module provides an interactive learning environment where users can:

- Select topic (e.g., property pricing, documentation, investment)
- Choose difficulty level
- Attempt generated questions

Output Displayed:

- Question with options
- Selected answer
- Score (correct/incorrect)

- Detailed explanation

This module helps users improve their understanding of real estate concepts through adaptive learning.

5.2. Brief Description of Various Modules of the System

The system is divided into multiple modules for efficient functioning:

Triage Agent Module

- Classifies messages based on urgency and intent
- Extracts entities using spaCy
- Generates draft responses using LLM
- Reduces manual workload in communication

Quiz Engine Module

- Generates domain-specific questions
- Evaluates user responses
- Provides explanations and scores
- Supports adaptive learning

API Management Module

- Handles all backend API requests
- Ensures structured JSON responses
- Manages communication between frontend and backend

Frontend Module

- Provides interactive UI
- Displays outputs clearly
- Handles user input and navigation

LLM Integration Module

- Performs classification, reasoning, and generation
- Ensures context-aware outputs
- Improves response quality

Deployment Module

- Manages containerization using Docker
- Automates deployment using Terraform and Ansible
- Ensures CI/CD using GitHub Actions

5.3. Snapshots of System

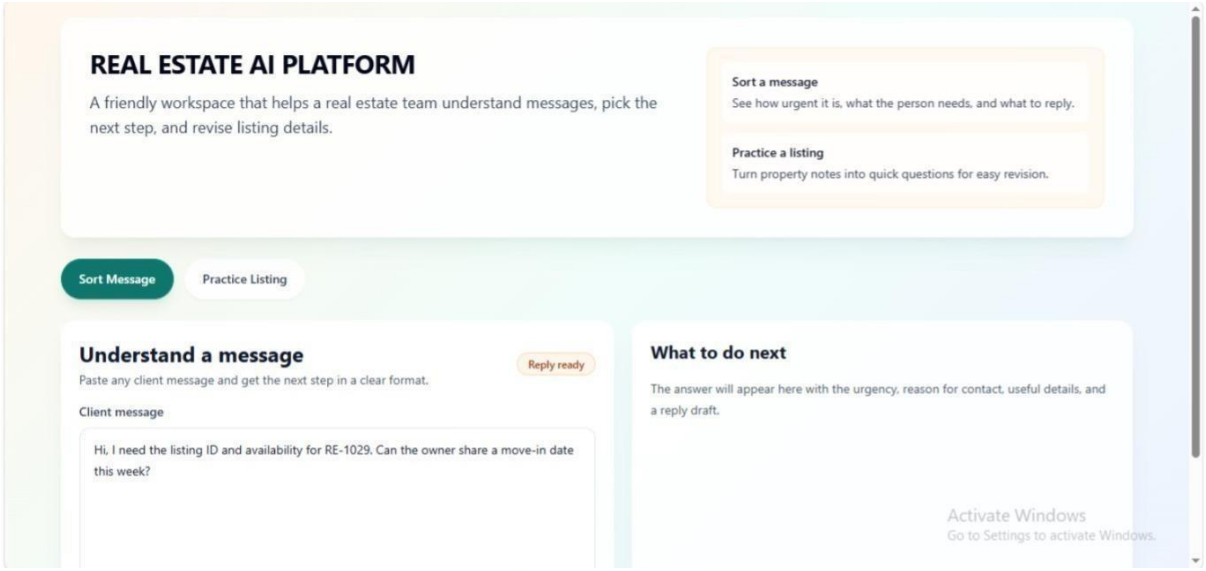

Figure 5.1 Screenshot of Frontend

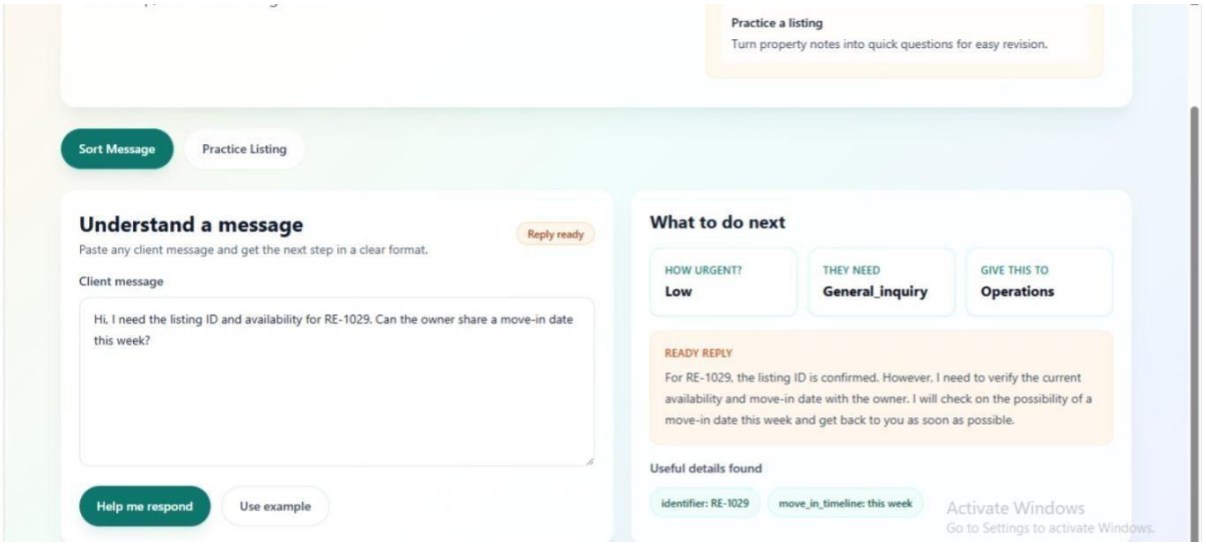

Figure 5.2 Screenshot of Inquiry & AI Response

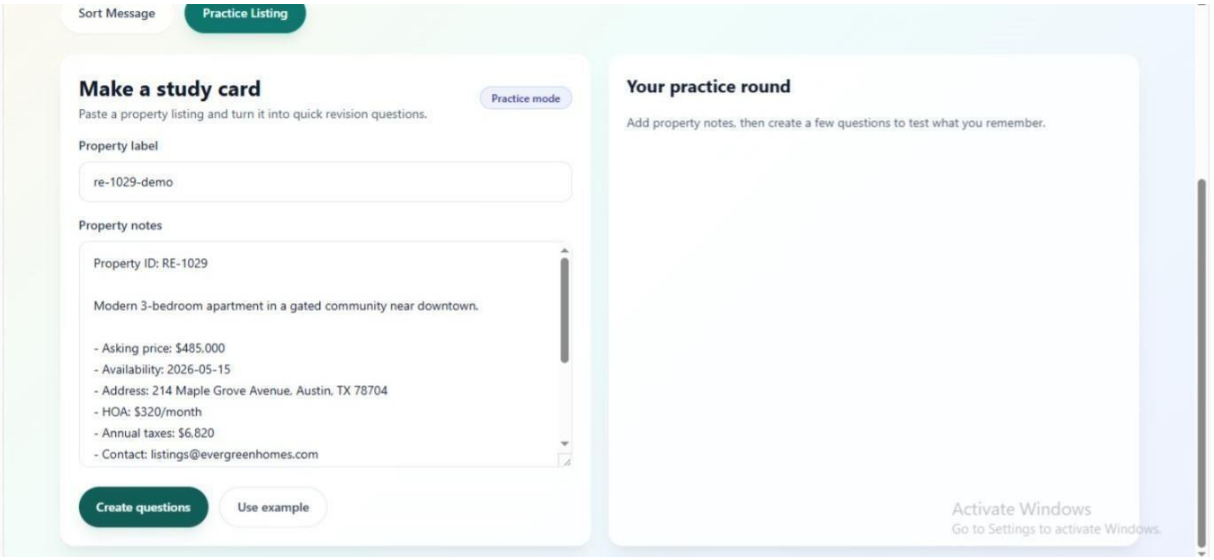

Figure 5.3 **Screenshot of QuizBot**

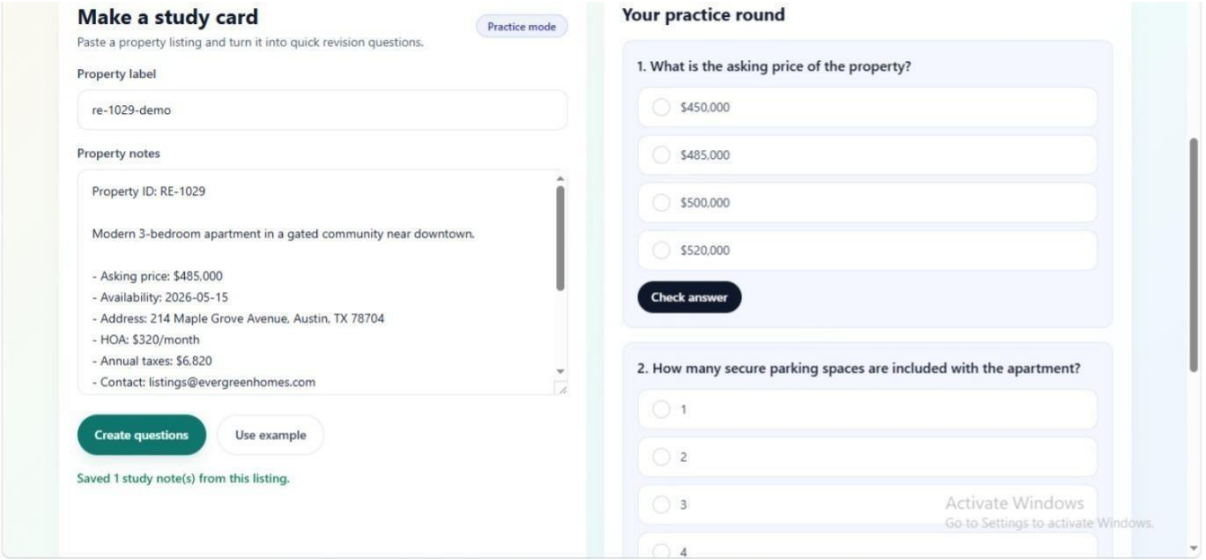

Figure 5.4 Screenshot of Quiz

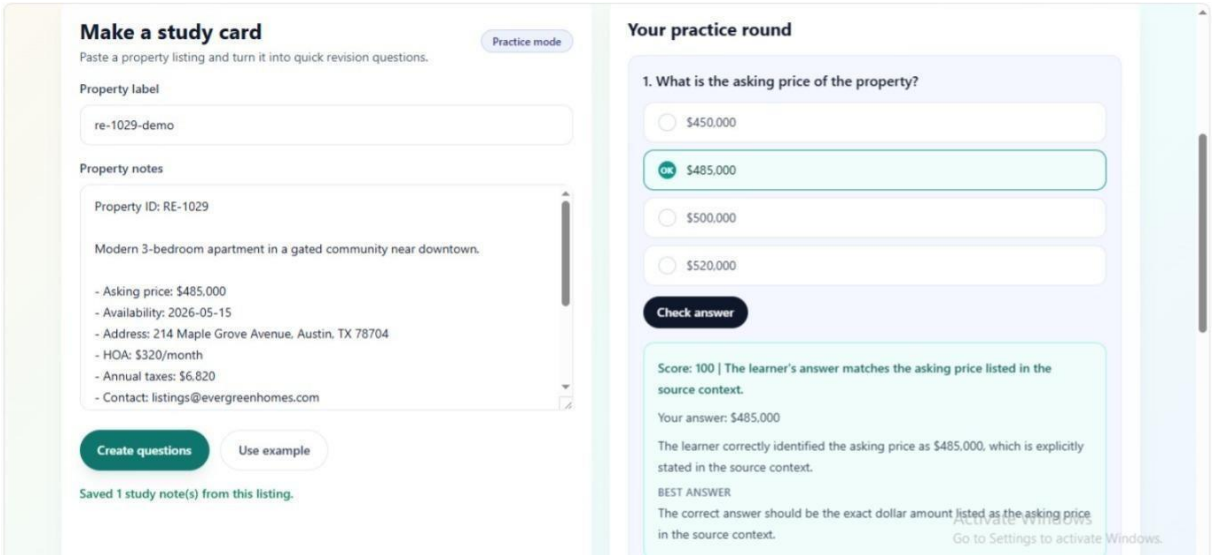

Figure 5.5 Screenshot of response & scoring

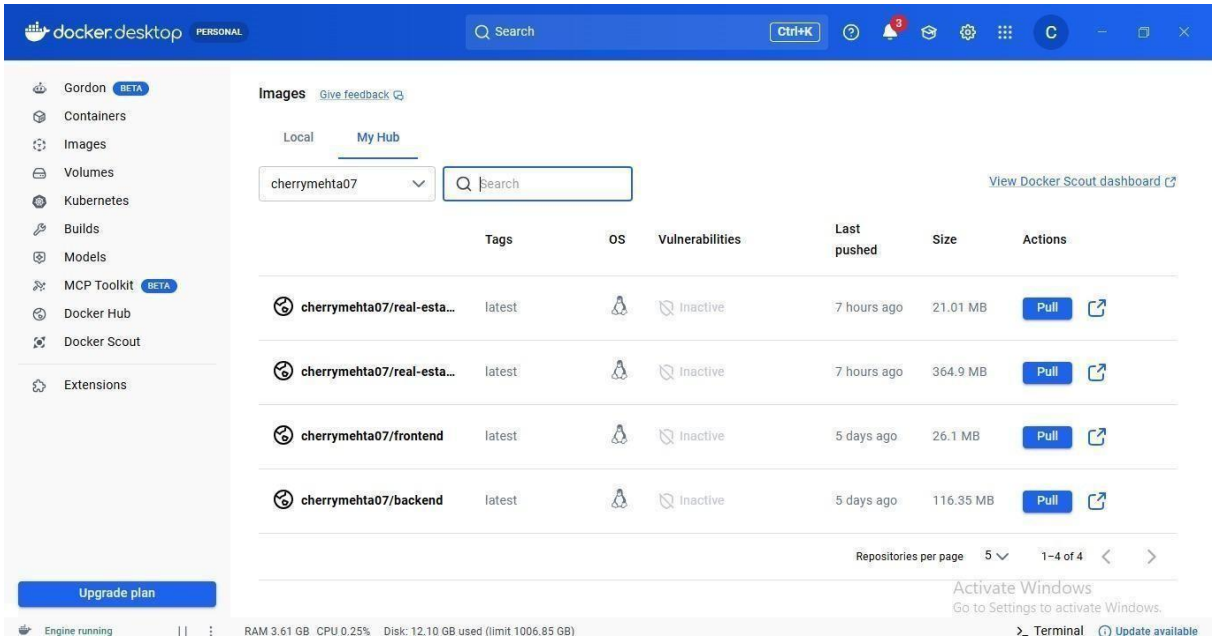

Figure 5.6 Docker Hub Images

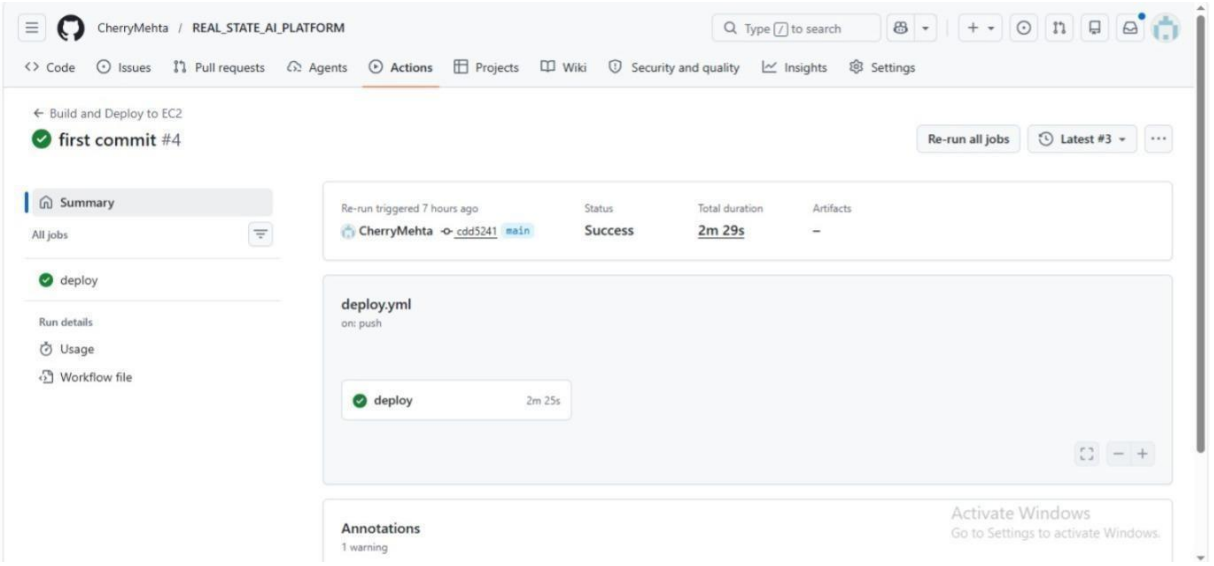

Figure 5.7 Deployment Successful

CHAPTER 6

SUMMARY AND CONCLUSIONS

6.1. Summary

The proposed **AI-Driven Real Estate Triage and Learning Platform** successfully demonstrates the integration of Artificial Intelligence, Natural Language Processing, and DevOps practices into a unified system designed to address real-world challenges in the Real Estate domain.

The system consists of two primary components:

- A **reactive triage agent** that automates the processing of real estate communications
- An **AI-driven educational bot** that enhances user learning through interactive assessments

The triage agent effectively classifies incoming messages based on urgency and intent, extracts key entities such as property IDs, pricing details, dates, and locations using Named Entity Recognition (NER), and generates context-aware draft responses using Large Language Models (LLMs). This significantly reduces manual workload and ensures faster, more consistent communication handling.

The educational module complements this functionality by generating scenario-based questions derived from property listings and real estate concepts. It evaluates user responses intelligently and provides detailed explanations, creating a personalized learning experience that improves domain knowledge and decision-making skills.

The system is built using a modern technology stack, with FastAPI for backend services and React (Vite) for frontend development. The integration of LLM APIs enables intelligent processing, while spaCy ensures efficient entity extraction. The entire application is containerized using Docker and deployed on AWS EC2, with infrastructure provisioning automated through Terraform and configuration managed using Ansible. CI/CD pipelines implemented using GitHub Actions ensure continuous integration and deployment.

Overall, the system achieves real-time performance, modular design, and scalable deployment, meeting all the objectives defined at the beginning of the project.

6.2. Conclusion

The developed system provides a comprehensive solution for automating communication workflows and enhancing learning in the Real Estate sector. The integration of Agentic AI and Generative AI enables intelligent processing of unstructured data, transforming traditional manual operations into efficient automated workflows.

The triage agent plays a crucial role in improving operational efficiency by quickly analyzing incoming messages, identifying intent and urgency, extracting relevant information, and

generating appropriate responses. This not only reduces response time but also improves customer experience and service quality.

The educational bot introduces an innovative approach to learning by combining AI-driven question generation with detailed feedback mechanisms. Unlike traditional quiz systems, it focuses on explanation-based learning, helping users understand concepts rather than just memorizing answers.

From a system design perspective, the project demonstrates a well-structured architecture with clear separation between frontend, backend, and AI components. The use of FastAPI ensures high-performance API handling, while React provides a responsive user interface. The integration of DevOps tools such as Docker, Terraform, Ansible, and GitHub Actions ensures scalability, reliability, and reproducibility of the system.

Despite certain limitations, such as dependency on external APIs and lack of persistent storage, the system proves to be effective, practical, and adaptable for real-world applications. It provides a strong foundation for future enhancements and large-scale deployment.

In conclusion, the project successfully achieves its goal of developing an intelligent, scalable, and user-centric platform that leverages AI to improve both operational efficiency and knowledge acquisition in the Real Estate domain.

CHAPTER 7

FUTURE SCOPE

7. Future Scope

- Integration of a persistent database to store real estate communication data, triage results, and quiz performance for improved data management and analytics.
- Implementation of user authentication and role-based access control to enable secure access for different users such as clients, agents, and administrators.
- Enhancement of the system using Retrieval-Augmented Generation (RAG) by integrating real estate datasets, property listings, and market trends to improve response accuracy and reduce dependency on generic LLM outputs.
- Support for multi-language processing to handle real estate communications in regional and international languages, improving accessibility for diverse users.
- Integration of analytics dashboards to monitor system performance, user activity, frequently asked queries, and response efficiency.
- Optimization of model performance to reduce response latency and improve cost efficiency for large-scale deployment.
- Extension of the system to support multi-modal inputs such as voice-based queries, document uploads (e.g., agreements), and image-based property analysis.
- Deployment on advanced cloud infrastructure with load balancing and auto-scaling to ensure high availability and scalability.
- Integration with real estate platforms and CRM systems to automate workflows such as property management, customer interaction, and transaction handling.

APPENDIX

Appendix A: Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
LLM	Large Language Model
NLP	Natural Language Processing
NER	Named Entity Recognition
RAG	Retrieval-Augmented Generation
API	Application Programming Interface
UI	User Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
CI/CD	Continuous Integration / Continuous Deployment
IaC	Infrastructure as Code
EC2	Elastic Compute Cloud
GPU	Graphics Processing Unit

BIBLIOGRAPHY

[1] T. Brown et al., "Language Models are Few-Shot Learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.

[2] OpenAI, "GPT-4 Technical Report," 2023.

[3] Meta AI, "LLaMA: Open and Efficient Foundation Language Models," 2023.

[4] Groq, "Groq API Documentation," [Online]. Available: https://console.groq.com

[5] OpenAI, "OpenAI API Documentation," [Online]. Available: https://platform.openai.com

[6] Explosion AI, "spaCy: Industrial-Strength Natural Language Processing," [Online]. Available: https://spacy.io

[7] CrewAI, "CrewAI Documentation," [Online]. Available: https://docs.crewai.com

[8] FastAPI, "FastAPI Framework Documentation," [Online]. Available: https://fastapi.tiangolo.com

[9] React, "React Documentation," [Online]. Available: https://react.dev

[10] Docker Inc., "Docker Documentation," [Online]. Available: https://docs.docker.com

[11] HashiCorp, "Terraform Documentation," [Online]. Available: https://developer.hashicorp.com/terraform

[12] Red Hat, "Ansible Documentation," [Online]. Available: https://docs.ansible.com

[13] Amazon Web Services, "Amazon EC2 Documentation," [Online]. Available: https://docs.aws.amazon.com/ec2

[14] GitHub, "GitHub Actions Documentation," [Online]. Available: https://docs.github.com/actions

[15] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *NeurIPS*, 2020.